



RAPPORT DE GÉNIE LOGICIEL ET CONDUITE DE PROJET

Application des concepts de Génie Logiciel
et de Conduite de Projet au développement de
SaaS Directory

Module : Génie Logiciel et Conduite de Projet

Encadrant : Pr. Zahid Nouredine

Présenté par :

Omar SARSAR

Iliass HARIZ

Soulaymane ELFAKIR

2025 / 2026

Dédicace

*Nous dédions ce travail à nos familles pour leur soutien indéfectible,
à nos enseignants pour leur guidance et leurs enseignements,
et à tous ceux qui croient en l'innovation technologique au service de la
communauté.*

Remerciements

Nous tenons à exprimer notre profonde gratitude envers l'ensemble de nos enseignants qui nous ont accompagnés tout au long de notre parcours académique. Leurs conseils avisés, leur patience et leur exigence ont été des facteurs déterminants dans la réussite de ce projet.

Nous remercions également nos camarades de promotion pour les échanges enrichissants et l'entraide collective qui ont contribué à notre progression.

Ce rapport présente l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de **SaaS Directory**, une plateforme de marketplace dédiée aux produits logiciels en mode SaaS. Le projet a été réalisé avec une stack technologique moderne comprenant Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL et Better Auth 1.6.9.

L'étude couvre l'ensemble des neuf chapitres du cours de Génie Logiciel : rappels fondamentaux, généralités, cycle de vie du logiciel, démarche de conception, normalisation, tests, gestion de projet, analyse des risques et conduite du changement. Chaque concept théorique est illustré par sa mise en oeuvre concrète dans le projet. Le rapport inclut une estimation du coût par la méthode COCOMO appliquée à la base de code réelle d'environ 11 000 lignes, ainsi qu'une présentation détaillée de la gestion des versions via Git et des migrations Drizzle ORM.

Mots-clés : Génie Logiciel, Conduite de Projet, Next.js, React, Drizzle ORM, PostgreSQL, CO-COMO, Cycle de Vie, SDLC, Scrum, Normalisation, Tests Logiciels, Gestion des Risques.

This report presents the systematic application of Software Engineering and Project Management concepts to the development of **SaaS Directory**, a marketplace platform dedicated to SaaS software products. The project was built using a modern technology stack including Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL, and Better Auth 1.6.9.

The study covers all nine chapters of the Software Engineering course : fundamental recalls, general concepts, software life cycle, design methodology, standardization, testing, project management, risk analysis, and change management. Each theoretical concept is illustrated by its concrete implementation in the project. The report includes a COCOMO cost estimation applied to the actual codebase of 12,166 lines, as well as a detailed presentation of version control via Git and Drizzle ORM migrations.

Keywords : Software Engineering, Project Management, Next.js, React, Drizzle ORM, PostgreSQL, COCOMO, Life Cycle, SDLC, Scrum, Standardization, Software Testing, Risk Management.

Dédicace	2
Remerciements	3
Résumé	4
Abstract	5
Introduction Générale	12
1 Chapitre 1 : Rappels Fondamentaux	13
1.1 Définitions Fondamentales	13
1.1.1 Logiciel et Application	13
1.1.2 Système d'Information (SI)	13
1.1.3 Méthode, Outil et Méthodologie	14
1.2 Application à SaaS Directory	14
2 Chapitre 2 : Généralités du Génie Logiciel	15
2.1 Objectifs du Génie Logiciel	15
2.2 Les Principes du GL Appliqués	16
2.2.1 Rigueur	16
2.2.2 Séparation des Problèmes	16
2.2.3 Modularité	16
2.2.4 Abstraction	18
2.2.5 Généricité	18
2.2.6 Construction Incrémentale	18
2.2.7 Anticipation du Changement	18
2.3 Les Attributs du Logiciel	18
2.3.1 Couplage	19
2.3.2 Complexité	19
2.3.3 Extensibilité	19

2.3.4	Lisibilité et Visibilité du Comportement	19
3	Chapitre 3 : Cycle de Vie du Logiciel (SDLC)	20
3.1	Les Étapes du SDLC	20
3.2	Les Modèles du SDLC	21
3.2.1	Modèle en Cascade (Waterfall)	21
3.2.2	Modèle en V	21
3.2.3	Modèle Agile	21
3.3	Application au Projet SaaS Directory	22
4	Chapitre 4 : Démarche de Conception	23
4.1	Le Framework Scrum	23
4.1.1	Rôles Scrum	24
4.1.2	Artefacts Scrum	24
4.1.3	Événements Scrum	24
4.2	Modélisation des Données — Méthode Merise	25
4.2.1	Modèle Conceptuel de Données (MCD)	25
4.2.2	Modèle Logique de Données (MLD)	26
4.3	Diagrammes UML Complémentaires	27
4.3.1	Diagramme de Cas d’Utilisation	27
4.3.2	Diagramme de Séquence	28
4.3.3	Diagramme d’Activité	28
4.3.4	Diagramme d’État	28
5	Chapitre 5 : La Normalisation	32
5.1	La Norme ISO et la Certification ISO 9000	32
5.2	Les Normes IEEE	33
5.3	Normalisation Appliquée au Projet	33
5.3.1	Normes de Codage	33
5.3.2	Architecture Normalisée	33
5.3.3	Gestion Documentaire et Traçabilité	34

6	Chapitre 6 : Les Tests	34
6.1	Types de Tests et Stratégie	34
6.1.1	Tests Unitaires	34
6.1.2	Tests End-to-End	35
6.1.3	Pyramide des Tests	35
6.1.4	Tests de Performance	36
6.2	Outils et Environnement de Test	37
6.3	Couverture et Résultats	37
7	Chapitre 7 : La Gestion des Projets Logiciels	37
7.1	Organisation du Projet (WBS, PBS, OBS)	38
7.2	Planification (GANTT)	38
7.3	Estimation des Coûts par COCOMO	39
7.3.1	Taille du Projet	39
7.3.2	Application du COCOMO de Base	40
8	Chapitre 8 : L'Analyse des Risques	40
8.1	Identification des Risques	40
8.1.1	Risques Techniques	40
8.1.2	Risques de Sécurité	41
8.1.3	Risques de Performance	41
8.2	Analyse et Stratégies de Mitigation	41
8.2.1	Mitigation des Risques Techniques	41
8.2.2	Mitigation des Risques de Sécurité	42
8.2.3	Mitigation des Risques de Performance	42
8.3	Plan de Continuité	42
9	Chapitre 9 : Conduite du Changement	43
9.1	Nature des Changements	43
9.2	Gestion des Versions	43
9.3	Adaptation et Communication	44

Conclusion Générale et Perspectives	45
Bibliographie et Webographie	46
Annexes	47
Annexe A : Dependances Principales	47
Annexe B : Diagramme de Deploiement	48

1	Architecture en Couches — SaaS Directory	17
2	Cycle de Vie du Logiciel (SDLC) — Application à SaaS Directory	22
3	Framework Scrum — Rôles, Artefacts et Événements	23
4	Modèle Conceptuel de Données (MCD) — SaaS Directory	26
5	Modèle Logique de Données (MLD) — SaaS Directory	27
6	Diagramme de Cas d'Utilisation — SaaS Directory	28
7	Diagramme de Séquence — Envoi d'un Message	29
8	Diagramme d'Activité — Flux d'Authentification	30
9	Diagramme d'État — Cycle de Vie des Abonnements	31
10	Pyramide des Tests — Stratégie de validation de SaaS Directory	35
11	Impact du Cache Redis sur les Performances — Comparaison avant/après	36
12	Diagramme de GANTT — Planification sur 60 Jours (à visée pédagogique)	39
13	Chronologie de l'Évolution du Schéma — Migrations Drizzle	44
14	Diagramme de Deploiement — Architecture Physique de SaaS Directory	48

1	Résultats du benchmark Cache Redis — SaaS Directory (10 mai 2026)	36
2	Estimation COCOMO de Base — Mode Semi-Détaché (14 KLOC)	40
3	Dependances principales du projet SaaS Directory	47

Dans un monde de plus en plus numérisé, le développement de logiciels est devenu une discipline complexe qui exige rigueur, méthodologie et gestion efficace des ressources. Le Génie Logiciel (GL) et la Conduite de Projet fournissent le cadre théorique et pratique nécessaire pour mener à bien des projets logiciels de qualité, dans les délais et budgets impartis.

Ce rapport a pour objectif de démontrer l'application concrète des concepts théoriques du Génie Logiciel et de la Conduite de Projet au développement de **SaaS Directory**, une application web moderne permettant aux créateurs de logiciels de lancer, promouvoir et vendre leurs produits SaaS.

SaaS Directory est une plateforme de marketplace dédiée aux produits logiciels en mode SaaS (Software as a Service). L'application offre un catalogue où les utilisateurs peuvent découvrir des outils logiciels, voter pour leurs produits préférés, interagir via des commentaires et une messagerie, et souscrire à des plans d'abonnement Free ou Pro.

Le projet a été réalisé avec une stack technologique contemporaine et cohérente : Next.js 16.2.4 (framework React avec App Router), React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2 (ORM type-safe pour PostgreSQL), Better Auth 1.6.9 (authentification complète avec OAuth et 2FA), Redis (cache côté serveur), et Playwright 1.59.1 pour les tests end-to-end.

Le codebase compte exactement **21 tables** dans le schéma Drizzle ORM, **environ 11 000 lignes de code** réparties sur **108 fichiers** TypeScript/TSX, avec **4 migrations** versionnées (0000 à 0003). Le projet est versionné sur Git avec une branche principale et des branches de fonctionnalités.

Ce rapport s'articule autour de neuf chapitres principaux. Le premier chapitre présente les rappels fondamentaux sur les concepts de base. Le deuxième traite des généralités du Génie Logiciel et de ses principes. Le troisième décrit les modèles de cycle de vie SDLC. Le quatrième expose la démarche de conception avec le framework Scrum. Le cinquième aborde la normalisation. Le sixième détaille les stratégies de tests. Le septième présente la gestion de projet et l'estimation CO-COMO. Le huitième analyse les risques. Le neuvième et dernier chapitre traite de la conduite du changement.

Introduction. Avant d’aborder les concepts avancés du Génie Logiciel et de la Conduite de Projet, il est essentiel de poser les bases terminologiques et conceptuelles. Ce chapitre présente les définitions fondamentales qui structurent l’ensemble du rapport : la notion de logiciel et d’application, le Système d’Information (SI) et le Système d’Information Automatisé (SIA), ainsi que la distinction entre méthode, outil et méthodologie. Ces concepts, bien que théoriques, conditionnent la compréhension des chapitres suivants et éclairent les choix techniques et organisationnels adoptés pour le projet SaaS Directory.

1.1 Définitions Fondamentales

1.1.1 Logiciel et Application

Un **logiciel** est défini comme un ensemble de programmes, procédures, règles et documentations relatifs au fonctionnement d’un ensemble de traitements d’informations. Cette définition, issue du vocabulaire international de l’ingénierie logicielle (ISO/IEC 24765), met en évidence le caractère multidimensionnel du logiciel : il ne se réduit pas au code source, mais englobe également la documentation, les procédures d’installation et de maintenance, ainsi que les règles de gestion associées.

Une **application** est un logiciel qui sert à implanter un système d’informations. Elle constitue l’interface concrète entre les utilisateurs et les données. On distingue trois grandes catégories d’applications : les applications de gestion (peu de traitement, beaucoup de données), les applications scientifiques (beaucoup de traitement, peu de données), et les applications industrielles (peu de traitement et de données). SaaS Directory s’inscrit dans la catégorie des applications de gestion/web, avec une composante significative de données structurées (21 tables relationnelles) et une logique métier modérée (marketplace, authentification, messagerie).

La distinction entre logiciel et application est cruciale pour le Génie Logiciel. Le logiciel est le produit final livré, tandis que l’application est la manifestation fonctionnelle de ce produit face aux utilisateurs. Pour SaaS Directory, le logiciel comprend l’ensemble du codebase (environ 11 000 lignes, 108 fichiers), les tests, la documentation technique et les scripts de déploiement. L’application, quant à elle, est la plateforme web accessible via navigateur qui permet aux créateurs de lancer leurs produits SaaS et aux utilisateurs de les découvrir.

1.1.2 Système d’Information (SI)

Le **Système d’Information (SI)** constitue l’ensemble des flux d’opérations, d’informations et de moyens mis en oeuvre par une organisation pour atteindre ses objectifs. Selon la définition classique, un SI comprend trois dimensions : la dimension organisationnelle (acteurs, rôles, processus), la dimension informatique (applications, données, infrastructures), et la dimension communication-

nelle (échanges, partage, collaboration).

Le **Système d’Information Automatisé (SIA)** repose sur la technologie informatique pour traiter, stocker et diffuser les informations. SaaS Directory est un SIA complet : il traite des profils utilisateurs, des produits, des votes, des messages et des abonnements via une infrastructure web moderne. L’automatisation touche l’ensemble du processus : création de compte, publication de produit, vote, commentaire, messagerie en temps réel, gestion des abonnements, et notification.

L’analyse du SI de SaaS Directory permet d’identifier les acteurs (visiteurs, utilisateurs authentifiés, créateurs de produits, administrateurs), les données manipulées (21 tables relationnelles), les traitements (Server Actions Next.js, cache Redis), et les flux d’information (HTTPS entre client et serveur, requêtes SQL via Drizzle ORM, lectures/écritures cache via Redis).

1.1.3 Méthode, Outil et Méthodologie

Une méthode est un processus discipliné qui génère un ensemble de modèles décrivant les différents aspects d’un système logiciel, en utilisant une notation bien définie.

Une méthode fournit le cadre méthodologique pour structurer le développement. Elle répond aux questions “comment” et “quand” : comment modéliser, comment tester, quand valider. Pour SaaS Directory, la méthode Merise a été utilisée pour la modélisation des données (MCD et MLD), tandis que les diagrammes UML ont servi à modéliser les aspects dynamiques et statiques du système.

Un **outil** désigne un programme ou un ensemble de programmes aidant à la mise en oeuvre des techniques. L’**Atelier de Génie Logiciel (AGL)** regroupe l’ensemble des outils utilisés pour le développement, la maintenance et la documentation. Pour ce projet, l’AGL comprend : Visual Studio Code (éditeur), Git (versioning), Tectonic (compilation LaTeX), PlantUML et matplotlib (diagrammes), Vitest et Playwright (tests), et les outils du navigateur (debugging).

La **méthodologie** est la démarche générale qui fait appel à des méthodes et des outils pour aboutir au logiciel final. Pour SaaS Directory, la méthodologie adoptée combine le cycle de vie SDLC (Analyse, Conception, Développement, Tests, Déploiement, Maintenance) avec le framework Scrum adapté au contexte académique (sprints courts de 3–4 jours, rôles implicites, livraison incrémentale).

1.2 Application à SaaS Directory

SaaS Directory s’inscrit dans le cadre théorique des Systèmes d’Information Automatisés. En tant que marketplace web, il traite de grands volumes de données structurées (21 tables, relations complexes many-to-many, clés étrangères polymorphes) et exige une architecture logicielle rigoureuse

pour garantir performance, sécurité et évolutivité.

La méthodologie adoptée pour ce projet est guidée par deux piliers. Le premier est le **cycle de vie SDLC** qui structure le développement en phases successives : analyse des besoins (spécifications fonctionnelles), conception (architecture, MCD/MLD, choix des technologies), développement (fondations puis fonctionnalités), tests (unitaires et end-to-end), préparation au déploiement (configuration Docker Compose local), et maintenance (migrations, corrections, évolutions). Le second pilier est le **framework Scrum** qui apporte l'agilité nécessaire pour s'adapter aux contraintes temporelles académiques et aux découvertes faites en cours de route.

Conclusion du chapitre. Ce chapitre a posé les bases terminologiques indispensables à la compréhension du rapport. Les notions de logiciel, d'application, de Système d'Information Automatisé, et la distinction entre méthode, outil et méthodologie éclairent les choix qui ont été faits pour le développement de SaaS Directory. Le projet s'inscrit dans la catégorie des applications de gestion web, avec une méthodologie hybride combinant SDLC et Scrum, et un AGL moderne comprenant des outils de développement, de modélisation et de test.

Introduction. Le Génie Logiciel (GL) constitue la discipline qui vise à produire des logiciels de qualité, dans les délais et budgets impartis, en appliquant des principes, des méthodes et des outils rigoureux. Ce chapitre présente les objectifs du GL, les principes fondamentaux qui le sous-tendent, et les attributs essentiels d'un logiciel bien conçu. Chaque principe et attribut est illustré par son application concrète dans le projet SaaS Directory, démontrant ainsi la pertinence de ces concepts théoriques dans un contexte de développement web moderne.

2.1 Objectifs du Génie Logiciel

Le **Génie Logiciel** vise à utiliser l'informatique dans les entreprises comme outil quotidien, à réduire les coûts de développement et de maintenance, à améliorer la qualité des logiciels, et à augmenter la productivité des équipes. Ces objectifs, formalisés dès les années 1970 par des pionniers comme Barry Boehm et Frederick Brooks, restent aujourd'hui d'actualité malgré l'évolution des technologies. Ils constituent le socle sur lequel reposent les décisions techniques et organisationnelles de tout projet logiciel, quel que soit son domaine ou sa taille.

Pour SaaS Directory, l'objectif de **réduction des coûts** s'est traduit par le choix d'une stack open-source complète (Next.js, React, TypeScript, Drizzle ORM, PostgreSQL, Redis) évitant les licences propriétaires coûteuses. Aucun coût de licence n'a été engagé, ce qui est particulièrement adapté à un contexte académique. L'objectif d'**amélioration de la qualité** a été assuré par le typage statique TypeScript, les conventions de nommage strictes (PascalCase pour les composants, camelCase pour

les fonctions), et la validation du code par l'équipe lors des sessions de travail collaboratif. L'objectif d'**augmentation de la productivité** a été facilité par l'utilisation de composants réutilisables (shaden/ui), l'automatisation des tests (Vitest, Playwright), et la génération automatique des migrations Drizzle qui éliminent le risque d'erreurs manuelles dans les scripts SQL.

Ces objectifs ne sont pas purement théoriques : ils se traduisent par des métriques concrètes sur le projet. Le codebase de environ 11 000 lignes a été maintenu avec une cohérence stylistique remarquée grâce aux conventions strictes. Les 10 fichiers de test automatisés garantissent une vérification systématique à chaque modification. La modularité du schéma Drizzle (21 tables distinctes) permet d'ajouter de nouvelles entités sans remettre en cause l'ensemble du système, ce qui constitue un indicateur tangible de productivité à long terme.

2.2 Les Principes du GL Appliqués

2.2.1 Rigueur

La rigueur se traduit par l'utilisation de TypeScript (typage statique strict), les conventions de nommage systématiques (PascalCase pour les composants, camelCase pour les fonctions, kebab-case pour les routes API), et la validation croisée du code au sein de l'équipe de trois développeurs. La documentation interne du projet (README et fichiers de configuration) documente les conventions spécifiques de développement. Cette rigueur a permis de détecter de nombreuses erreurs à la compilation plutôt qu'à l'exécution, réduisant considérablement le temps de débogage.

2.2.2 Séparation des Problèmes

Trois niveaux de séparation sont appliqués : séparation dans le temps (développement par phases avec tests à chaque niveau), séparation dans l'espace (architecture en couches, voir figure 1), et séparation en sous-systèmes (modules application, composants et utilitaires serveur).

L'architecture en couches de SaaS Directory (figure 1) illustre parfaitement ce principe. La couche client (navigateur, React, composants UI) est séparée de la couche serveur (Next.js, App Router, Server Actions), elle-même séparée de la couche d'accès aux données (Drizzle ORM, requêtes type-safe) et de la couche persistance (PostgreSQL, Redis Cache). Chaque couche communique avec la suivante via des interfaces bien définies (HTTPS, ORM, SQL/Cache).

2.2.3 Modularité

L'architecture repose sur des composants React indépendants, chacun responsable d'une fonctionnalité spécifique. Le schéma Drizzle ORM décompose le système en 21 tables distinctes, chacune encapsulant une entité métier. Cette modularité facilite la maintenance (modification d'une table sans impacter les autres), les tests unitaires (test de chaque composant isolément), et l'évolutivité

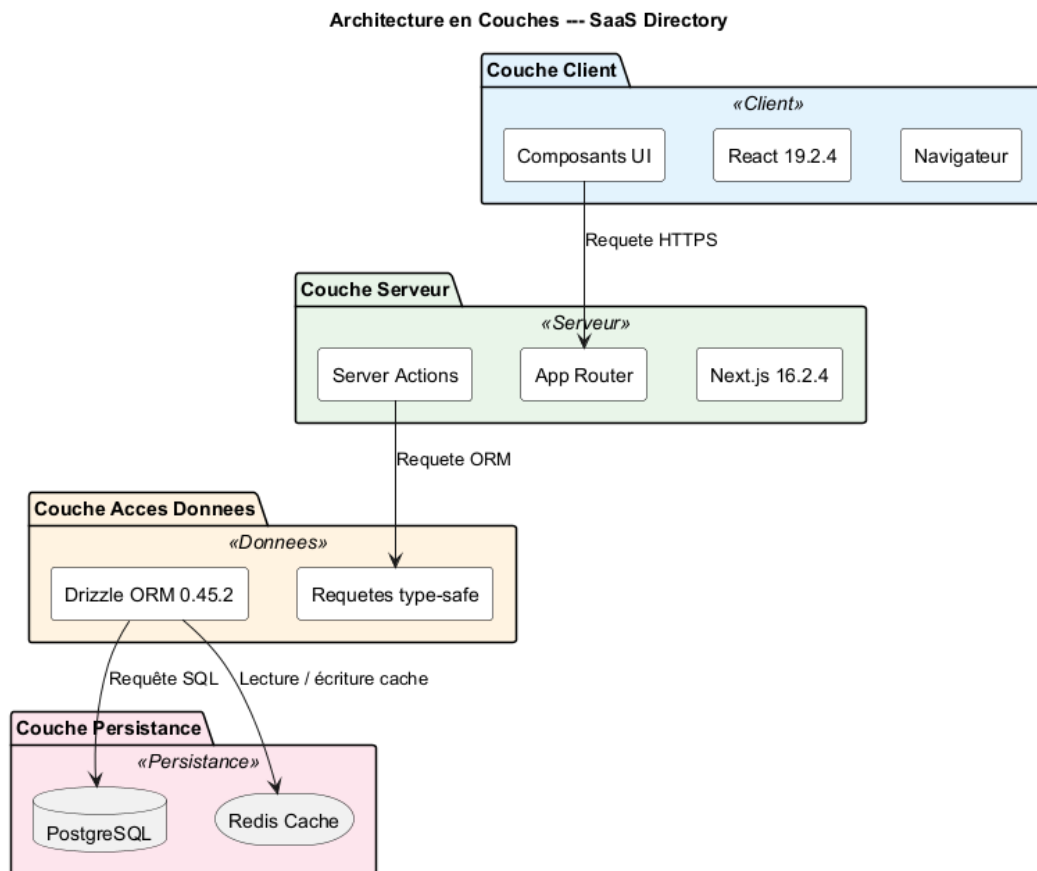


FIGURE 1 – Architecture en Couches — SaaS Directory

(ajout de nouvelles tables sans refactoring majeur).

2.2.4 Abstraction

Plusieurs niveaux d'abstraction sont utilisés : les Server Actions abstraient la logique métier du rendu UI ; Drizzle ORM abstrait les requêtes SQL complexes via une API type-safe ; le système de cache Redis abstrait la gestion des données temporaires. L'abstraction permet aux développeurs de travailler à un niveau de conception élevé sans se soucier des détails d'implémentation sous-jacents.

2.2.5 Généricité

Les composants UI sont conçus comme des composants réutilisables paramétrables via les propriétés React. Le système d'authentification Better Auth est générique et supporte OAuth (Google, GitHub) et l'authentification par email/mot de passe. La généricité évite la duplication de code et favorise la cohérence visuelle et fonctionnelle de l'application.

2.2.6 Construction Incrémentale

Le développement a suivi une approche incrémentale documentée dans l'historique Git. Les phases principales sont : fondations (Next.js 16.2.4, shadcn, Drizzle), infrastructure de tests (Vitest, Playwright), base de données (schéma Drizzle, migrations, seed), authentification TDD (login/signup, tests), fonctionnalités core (marketplace, profils, messages, notifications), et polissage final (recherche, responsive, build fixes). Chaque incrément livrait une version fonctionnelle testable, conforme au principe des livrables du framework Scrum.

2.2.7 Anticipation du Changement

L'architecture anticipe plusieurs évolutions : le schéma Drizzle inclut déjà les champs client et abonnement Stripe pour une future intégration du paiement ; le système de plans (Free/Pro) permet d'ajouter de nouveaux niveaux d'abonnement sans modification structurelle. L'anticipation du changement est un principe clé du GL qui évite les refactorings coûteux à long terme.

2.3 Les Attributs du Logiciel

L'analyse du codebase de environ 11 000 lignes réparties sur 108 fichiers permet d'évaluer ces attributs de manière quantitative et qualitative.

2.3.1 Couplage

Le couplage est faible grâce à l'architecture par composants React et l'encapsulation des Server Actions. Chaque table Drizzle ORM est indépendante, avec des relations explicites via les clés étrangères. Le couplage entre frontend (React) et backend (Server Actions) est réduit au minimum par l'API interne de Next.js 16.2.4. Un faible couplage signifie que la modification d'un module n'impacte que marginalement les autres modules.

2.3.2 Complexité

Avec 21 tables relationnelles et environ 11 000 lignes de code, le système présente une complexité modérée. Cette complexité est maîtrisée par la modularité (composants React réutilisables, Server Actions encapsulées), le typage statique TypeScript qui détecte les erreurs à la compilation, et les interfaces clairement nommées qui documentent implicitement les contrats de chaque fonction. L'absence de fonctions monolithiques et la décomposition systématique en petites unités de traitement garantissent que la complexité reste contrôlable pour les trois développeurs du projet.

2.3.3 Extensibilité

L'extensibilité est démontrée par la facilité d'ajouter de nouvelles tables au schéma Drizzle (4 migrations versionnées), l'ajout de nouveaux composants UI sans impacter les existants, et la configuration modulaire du framework. L'ajout du système de notifications (migration 0003) n'a nécessité que la modification de la table Profile et la création de handlers dédiés, illustrant une extensibilité réelle.

2.3.4 Lisibilité et Visibilité du Comportement

La lisibilité est assurée par TypeScript (typage explicite qui sert de documentation vivante), les conventions de nommage strictes (PascalCase pour les composants, camelCase pour les fonctions, kebab-case pour les routes), et les interfaces et types clairement nommés dans le schéma Drizzle. La visibilité du comportement est garantie par les tests unitaires (10 fichiers de test couvrant l'authentification, la souscription, le schéma, les actions serveur) et les tests end-to-end (Playwright 1.59.1). Chaque fonction critique possède au moins un test associé, assurant une visibilité complète du comportement attendu.

Conclusion du chapitre. Ce chapitre a présenté les objectifs, les principes et les attributs du Génie Logiciel, en illustrant chaque concept par son application concrète dans le projet SaaS Directory. La rigueur, la séparation des problèmes, la modularité, l'abstraction, la généralité, la construction incrémentale et l'anticipation du changement constituent les fondements méthodologiques du projet. Les attributs de couplage, complexité, extensibilité et lisibilité démontrent la qualité architecturale du livrable.

Introduction. Le développement d'un logiciel n'est pas simplement l'écriture d'un programme. La taille et la complexité des logiciels actuels exigent l'utilisation de processus de développement structurés, appelés **cycles de vie du logiciel** ou **Software Development Life Cycle (SDLC)**. Ce chapitre présente les six étapes fondamentales du SDLC, les trois modèles principaux (Waterfall, V, Agile), et leur application au projet SaaS Directory.

3.1 Les Étapes du SDLC

Le SDLC décrit les phases par lesquelles passe tout projet logiciel, de la naissance de l'idée à la maintenance en passant par le retrait éventuel. Ces étapes constituent un cadre de référence universel applicable à tous les types de projets logiciels, quelle que soit leur taille ou leur domaine.

1. Analyse des besoins. Cette première phase consiste à identifier, documenter et valider les besoins des utilisateurs et des parties prenantes. Elle produit un cahier des charges fonctionnel détaillant les fonctionnalités attendues, les contraintes techniques, et les critères d'acceptation. Pour SaaS Directory, l'analyse des besoins a identifié les rôles (visiteur, utilisateur, créateur, admin), les fonctionnalités (catalogue, vote, commentaire, messagerie, abonnement), et les contraintes (performance, sécurité, responsive).

2. Conception. La conception traduit les besoins en spécifications techniques. Elle comprend la conception architecturale (choix de la stack, architecture en couches), la conception de la base de données (MCD et MLD Merise, schéma Drizzle), et la conception de l'interface utilisateur (composants React modulaires basés sur shadcn/ui). En raison des contraintes temporelles du projet académique, aucune maquette formelle (wireframe) n'a été produite ; les décisions d'interface ont été prises directement lors du développement des composants. La conception de SaaS Directory a abouti au choix de Next.js 16.2.4 avec App Router, Drizzle ORM 0.45.2, PostgreSQL, et Redis.

3. Développement. Le développement est la phase de codage proprement dite. Elle suit les spécifications de conception et utilise les outils et méthodes définis. Pour SaaS Directory, le développement s'est déroulé en deux sous-phases : les fondations (setup projet, configuration ORM, authentification) et les fonctionnalités core (marketplace, profils, messagerie, notifications). Le code source totalise environ 11 000 lignes réparties sur 108 fichiers TypeScript/TSX.

4. Tests. Les tests visent à détecter les erreurs et à garantir que le système répond aux exigences spécifiées. Ils comprennent les tests unitaires (composants isolés), les tests d'intégration (interactions entre modules), et les tests de validation (conformité aux besoins). SaaS Directory utilise Vitest pour les tests unitaires et Playwright pour les tests end-to-end. Le benchmark Redis mesure également les performances.

5. Déploiement. Le déploiement consiste à installer le logiciel dans l'environnement de production et à le rendre accessible aux utilisateurs. Il inclut la configuration des serveurs, la mise en place

de la base de données, et la publication de l'application. Pour ce projet académique, la préparation au déploiement a été documentée via un fichier Docker Compose définissant les services Next.js, PostgreSQL et Redis pour un environnement de développement local standardisé. Un déploiement en production sur un hébergeur cloud n'a pas été réalisé dans le cadre du projet académique, mais la configuration Docker constitue une base prête pour une future mise en production.

6. Maintenance. La maintenance couvre les corrections d'anomalies, les évolutions fonctionnelles, et l'optimisation des performances. Elle s'étend sur toute la durée de vie du logiciel. Pour SaaS Directory, la maintenance est assurée par les migrations Drizzle versionnées (0000 à 0003), le versioning Git, et la documentation technique.

3.2 Les Modèles du SDLC

3.2.1 Modèle en Cascade (Waterfall)

Le modèle en cascade, proposé par Winston Royce en 1970, suit une séquence linéaire de phases : chaque étape doit être entièrement terminée avant que la suivante ne commence. Ce modèle est adapté aux projets où les besoins sont parfaitement définis dès le départ et peu susceptibles d'évoluer. Ses avantages sont la simplicité de gestion et la traçabilité des exigences. Ses inconvénients sont la rigidité face aux changements et la détection tardive des erreurs (souvent lors des tests, en fin de cycle).

Pour SaaS Directory, un pur modèle en cascade aurait été inadapté car les besoins ont évolué en cours de route (ajout de Redis, modifications du schéma). Cependant, l'esprit séquentiel du waterfall a été conservé pour la planification globale du projet.

3.2.2 Modèle en V

Le modèle en V associe à chaque phase de développement une phase de test correspondante. La branche descendante du V représente les activités de conception et de codage, tandis que la branche montante représente les activités de vérification et de validation. Ce modèle garantit que chaque niveau de conception est testé avant de passer au niveau supérieur.

SaaS Directory a intégré des éléments du modèle en V : chaque fonctionnalité développée (branche descendante) a fait l'objet de tests unitaires ou end-to-end (branche montante) avant d'être intégrée à la branche principale. Cette approche a permis de détecter rapidement les régressions.

3.2.3 Modèle Agile

Le modèle Agile privilégie l'itération, la collaboration avec le client, et l'adaptation au changement. Il divise le projet en itérations courtes (sprints de 1 à 4 semaines), chacune livrant une version fonctionnelle du logiciel. Le framework Scrum est l'implémentation la plus répandue du modèle

Agile.

SaaS Directory a adopté une version adaptée du modèle Agile. Les sprints de 3 à 4 jours ont permis de livrer des incréments fonctionnels testables (authentification, puis marketplace, puis messagerie). Cette flexibilité a été essentielle dans un contexte académique où les contraintes évoluent rapidement.

3.3 Application au Projet SaaS Directory

Le développement de SaaS Directory s’est déroulé sur environ trois semaines, du 5 au 21 mai 2026. Le travail a été réparti entre trois développeurs selon une approche inspirée du SDLC et du Scrum. Le diagramme de la figure 2 illustre le cycle de vie adopté.

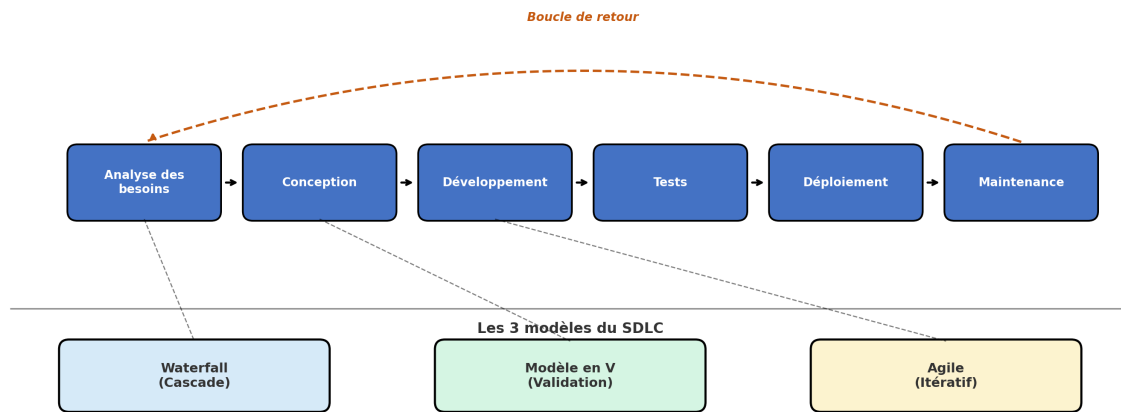


FIGURE 2 – Cycle de Vie du Logiciel (SDLC) — Application à SaaS Directory

Phase 1 : Analyse des besoins (jours 1–3). Identification des acteurs, des fonctionnalités, et des contraintes. Rédaction des user stories et définition du Product Backlog initial.

Phase 2 : Conception (jours 4–6). Choix de la stack technique, modélisation Merise (MCD et MLD), architecture en couches, définition des composants UI réutilisables.

Phase 3 : Développement — Fondations (jours 7–10). Initialisation du projet Next.js 16.2.4, configuration de Drizzle ORM avec PostgreSQL, mise en place de Better Auth, création des tables de base.

Phase 4 : Développement — Fonctionnalités Core (jours 11–15). Authentification, marketplace (votes, commentaires), messagerie, abonnements.

Phase 5 : Tests (jours 16–18). Tests unitaires Vitest, tests end-to-end Playwright, benchmark performance Redis.

Phase 6 : Déploiement et Maintenance (jours 19–21). Build, corrections, documentation, rapport.

Conclusion du chapitre. Ce chapitre a présenté le cycle de vie du logiciel (SDLC) et ses trois modèles principaux. Le projet SaaS Directory a adopté une approche hybride : la structure globale du SDLC (6 étapes) pour la planification, et l’esprit itératif du modèle Agile (via une adaptation de Scrum) pour l’exécution. Cette combinaison a permis de maintenir une vision structurée tout en conservant la flexibilité nécessaire à un contexte académique intensif.

Introduction. La démarche de conception d’un logiciel définit la méthodologie suivie pour passer du besoin exprimé au logiciel opérationnel. Ce chapitre présente le framework Scrum, méthode agile étudiée dans le cours, et son application au projet SaaS Directory. Il détaille également la modélisation des données (MCD et MLD selon la méthode Merise) et les diagrammes UML complémentaires (cas d’utilisation, séquence, activité, état, déploiement) qui documentent le système.

4.1 Le Framework Scrum

Scrum est un framework agile de gestion de projets qui organise le développement en sprints itératifs. Il repose sur trois piliers : la transparence, l’inspection et l’adaptation. Le diagramme de la figure 3 présente les éléments fondamentaux du framework.

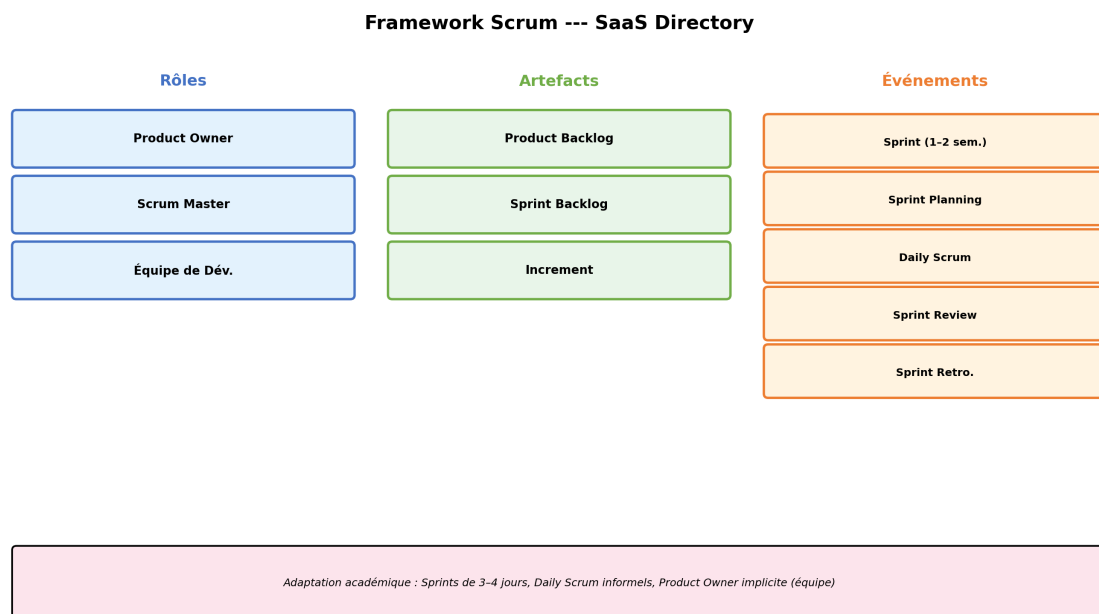


FIGURE 3 – Framework Scrum — Rôles, Artefacts et Événements

4.1.1 Rôles Scrum

Le **Product Owner** est responsable de la vision du produit et de la priorisation du Product Backlog. Il représente les intérêts des parties prenantes et maximise la valeur du produit. Dans le contexte académique de SaaS Directory, le rôle de Product Owner a été implicite : l'équipe elle-même a défini les priorités en fonction des objectifs pédagogiques et des contraintes temporelles.

Le **Scrum Master** facilite le processus Scrum, élimine les obstacles, et garantit l'adhésion aux pratiques agiles. Il ne dirige pas l'équipe mais la coach. Pour ce projet, le rôle de Scrum Master a été partagé entre les trois développeurs de manière informelle, chacun prenant l'initiative de lever les blocages rencontrés.

L'**Équipe de Développement** est constituée de trois développeurs (Omar SARSAR, Iliass HARIZ, Soulaymane ELFAKIR). L'équipe est auto-organisée et cross-fonctionnelle : chaque membre a contribué au frontend, au backend, et à la base de données selon ses compétences et les besoins du sprint.

4.1.2 Artefacts Scrum

Le **Product Backlog** est la liste ordonnée de toutes les fonctionnalités souhaitées pour le produit. Pour SaaS Directory, le Product Backlog initial comprenait : authentification (OAuth + email), profils utilisateurs, catalogue de produits, système de votes, commentaires, messagerie, abonnements Free/Pro, et tableau de bord administrateur.

Le **Sprint Backlog** est le sous-ensemble du Product Backlog sélectionné pour un sprint donné, décomposé en tâches techniques. Par exemple, le sprint "Authentification" a décomposé la user story en tâches : configuration Better Auth, pages login/signup, tests unitaires, gestion des sessions.

L'**Increment** est la somme de tous les éléments du Product Backlog complétés pendant un sprint, plus les increments des sprints précédents. Chaque increment doit être fonctionnel et testé. Pour SaaS Directory, les increments livrés ont été : (1) authentification fonctionnelle, (2) profils + marketplace basique, (3) votes + commentaires, (4) messagerie, (5) abonnements + polissage.

4.1.3 Événements Scrum

Le **Sprint** est une période fixe (généralement 1 à 4 semaines) pendant laquelle un increment est créé. Pour SaaS Directory, les sprints ont duré 3 à 4 jours en raison des contraintes académiques.

Le **Sprint Planning** lance chaque sprint en définissant ce qui sera réalisé et comment. L'équipe sélectionne les user stories du Product Backlog et les décompose en tâches. Dans le contexte académique du projet, ces planifications étaient informelles et intégrées aux sessions de travail collectif.

Le **Daily Scrum** est théoriquement un stand-up quotidien de 15 minutes où chaque membre répond à trois questions : qu'ai-je fait hier ? Que vais-je faire aujourd'hui ? Quels obstacles rencontre-je ?

Dans le contexte du projet, les points de synchronisation étaient informels et adaptés au rythme des travaux pratiques, sans la formalité d'un stand-up strict.

Le **Sprint Review** présente l'increment aux parties prenantes et recueille le feedback. Pour ce projet, les validations ont eu lieu à la fin de chaque phase majeure avec des démonstrations fonctionnelles entre les membres de l'équipe.

Le **Sprint Retrospective** examine le processus du sprint et identifie les améliorations. L'équipe a utilisé des moments de réflexion informels pour ajuster sa méthode de travail, par exemple en identifiant la nécessité d'ajouter Redis pour le cache après avoir constaté des problèmes de performance lors des tests.

4.2 Modélisation des Données — Méthode Merise

La méthode Merise, développée en France dans les années 1970, est une méthode de conception de systèmes d'information qui sépare les données des traitements. Elle propose trois niveaux de modélisation : le MCD (Modèle Conceptuel de Données), le MLD (Modèle Logique de Données), et le MPD (Modèle Physique de Données).

4.2.1 Modèle Conceptuel de Données (MCD)

Le MCD représente les entités du système, leurs attributs, et les associations entre elles avec les cardinalités. Il est indépendant de toute technologie. Le diagramme de la figure 4 présente le MCD de SaaS Directory.

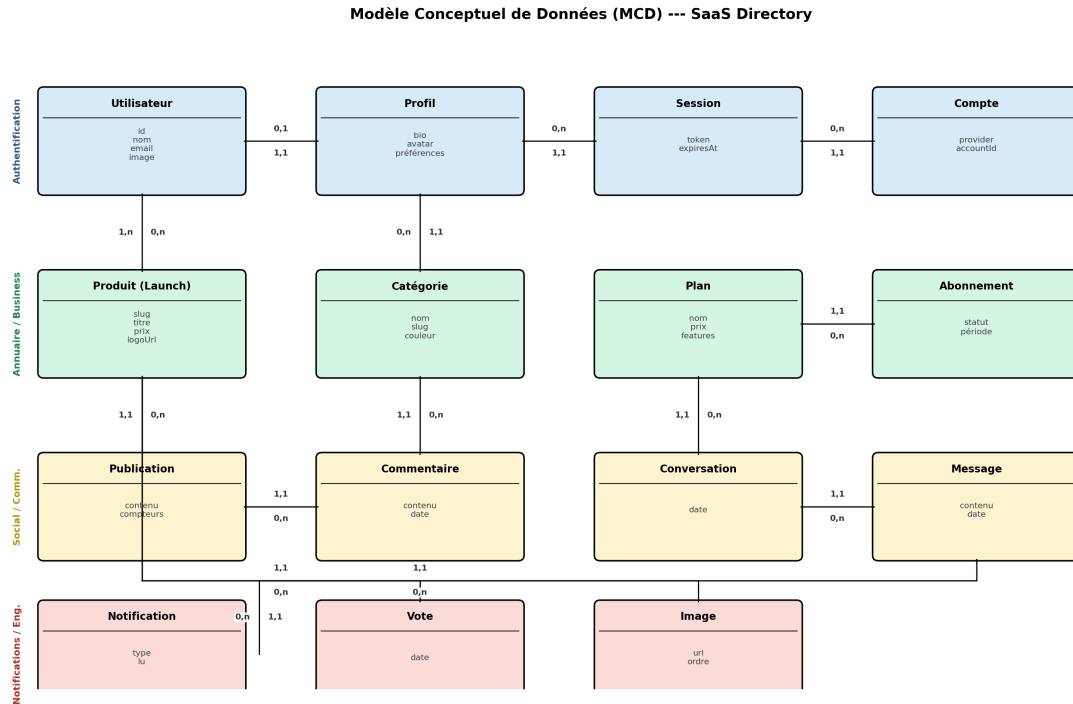


FIGURE 4 – Modèle Conceptuel de Données (MCD) — SaaS Directory

Le MCD de SaaS Directory compte 15 entités regroupées en 4 domaines fonctionnels. Le domaine **Authentification** comprend les entités Utilisateur, Profil, Session et Compte. Le domaine **Annuaire** comprend les entités Produit, Catégorie, Plan et Abonnement. Le domaine **Social** comprend les entités Publication, Commentaire, Conversation et Message. Le domaine **Engagement** comprend les entités Notification, Vote et Image.

Les cardinalités illustrent les règles métier : un Utilisateur possède un et un seul Profil (1,1), mais peut créer plusieurs Produits (0,n). Un Produit appartient à plusieurs Catégories (0,n) via une association many-to-many matérialisée par la table launchCategory. Ces cardinalités garantissent l'intégrité sémantique des données.

4.2.2 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en un modèle relationnel. Les entités deviennent des tables, les attributs deviennent des colonnes, et les associations sont traduites en clés étrangères selon les règles de passage Merise. Le diagramme de la figure 5 présente le MLD de SaaS Directory.

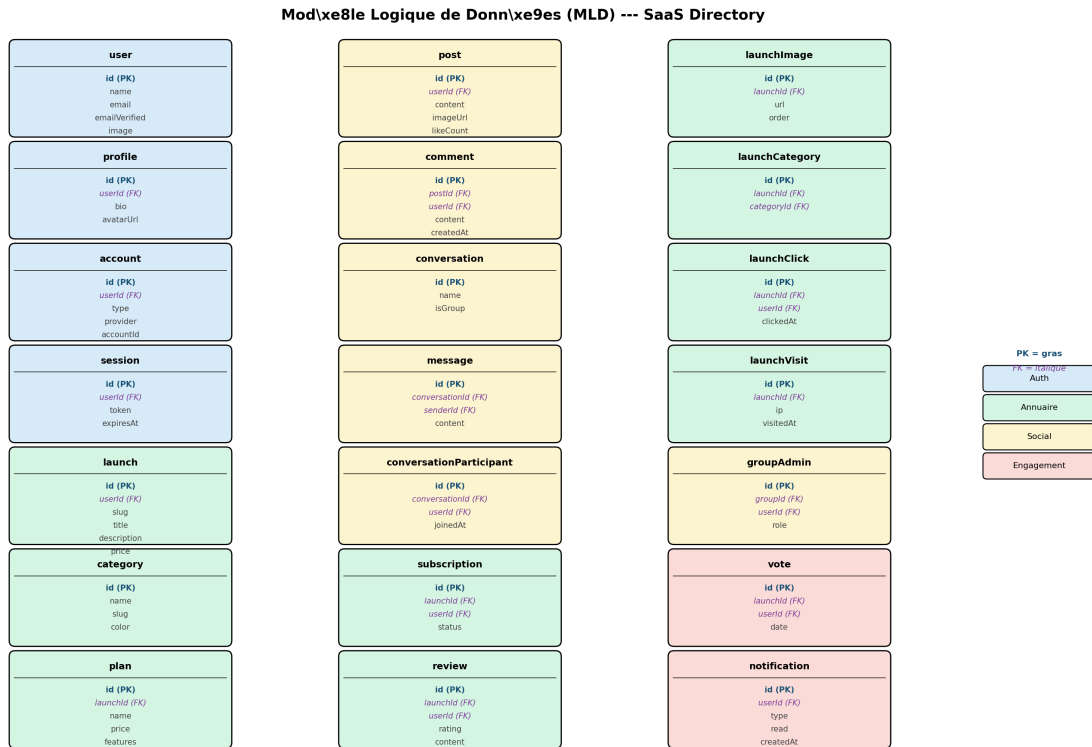


FIGURE 5 – Modèle Logique de Données (MLD) — SaaS Directory

Le MLD de SaaS Directory compte 21 tables. Les clés primaires (PK) sont soulignées en gras. Les clés étrangères (FK) sont indiquées en italique. Les associations many-to-many (par exemple Utilisateur-Produit pour les votes) sont matérialisées par des tables de liaison avec une clé primaire composite (userId, launchId).

Le passage du MCD au MLD suit les règles strictes de Merise : une association 1,n devient une clé étrangère côté “n” ; une association 0,n se traduit également par une clé étrangère avec contrainte nullable ; une association many-to-many nécessite une table intermédiaire. Ce MLD est ensuite implémenté directement par le schéma Drizzle ORM du projet.

4.3 Diagrammes UML Complémentaires

En complément de la modélisation Merise, plusieurs diagrammes UML ont été produits pour documenter les aspects dynamiques et structurels du système.

4.3.1 Diagramme de Cas d’Utilisation

Le diagramme de cas d’utilisation modélise les interactions entre les acteurs et le système. Les acteurs principaux sont : Visiteur, Utilisateur Authentifié, Créateur de Produit, et Administrateur.

Diagramme de Cas d'Utilisation --- SaaS Directory

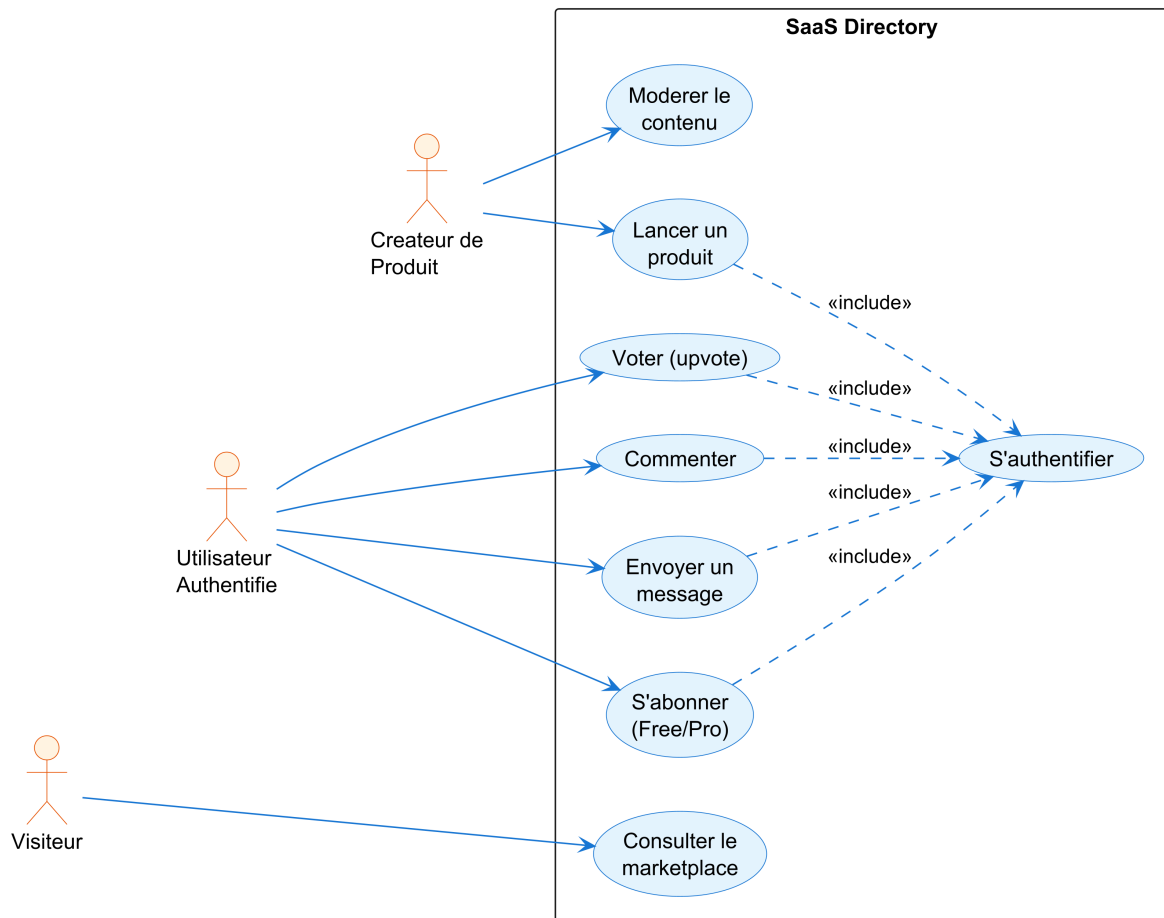


FIGURE 6 – Diagramme de Cas d'Utilisation — SaaS Directory

4.3.2 Diagramme de Séquence

Le diagramme de séquence illustre l'interaction entre les acteurs et le système lors de l'envoi d'un message.

L'utilisateur émetteur saisit le message via l'interface React. Le composant client appelle l'action serveur qui insère le message dans la base de données via l'ORM. Cette architecture démontre la séparation des préoccupations entre présentation, métier et persistance.

4.3.3 Diagramme d'Activité

Le diagramme d'activité de la figure 8 illustre le flux complet d'authentification.

4.3.4 Diagramme d'État

Le diagramme de la figure 9 présente le cycle de vie des abonnements.

Conclusion du chapitre. Ce chapitre a présenté la démarche de conception du projet SaaS Di-

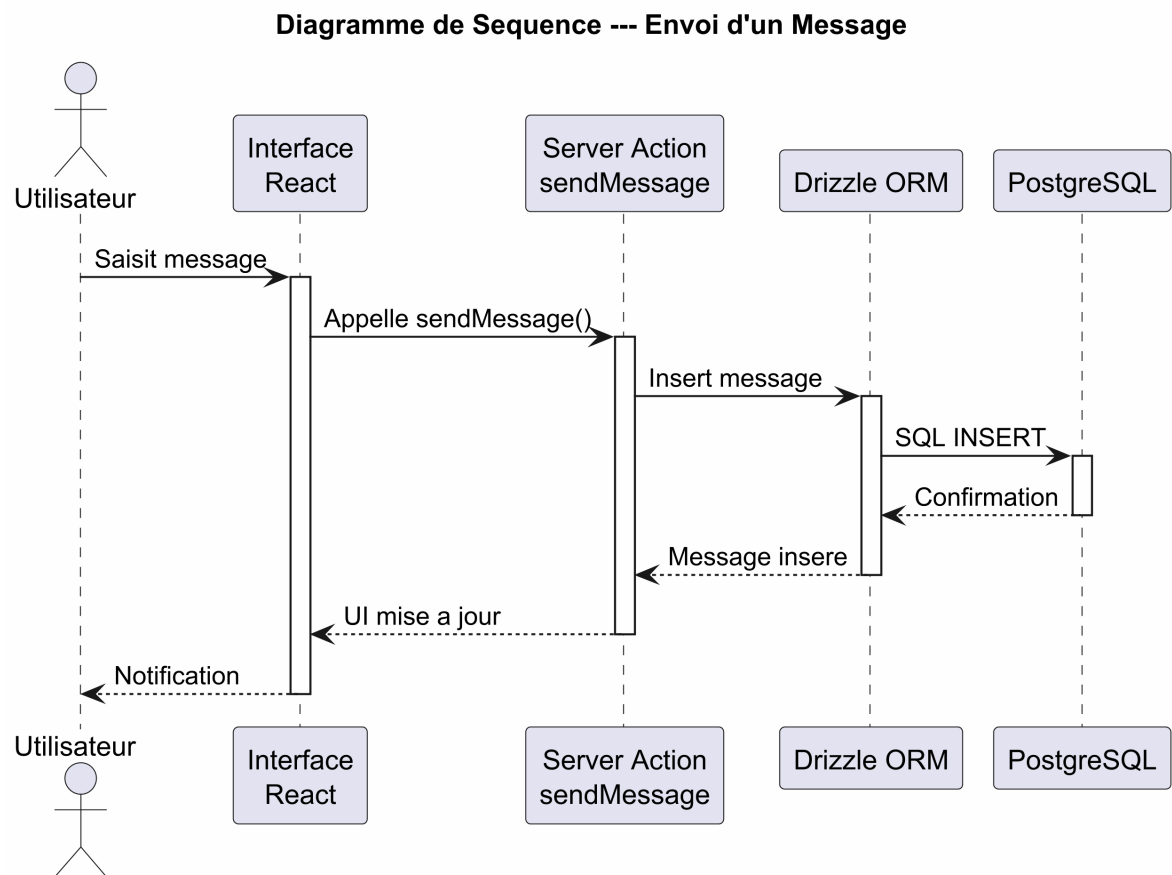


FIGURE 7 – Diagramme de Séquence — Envoi d'un Message

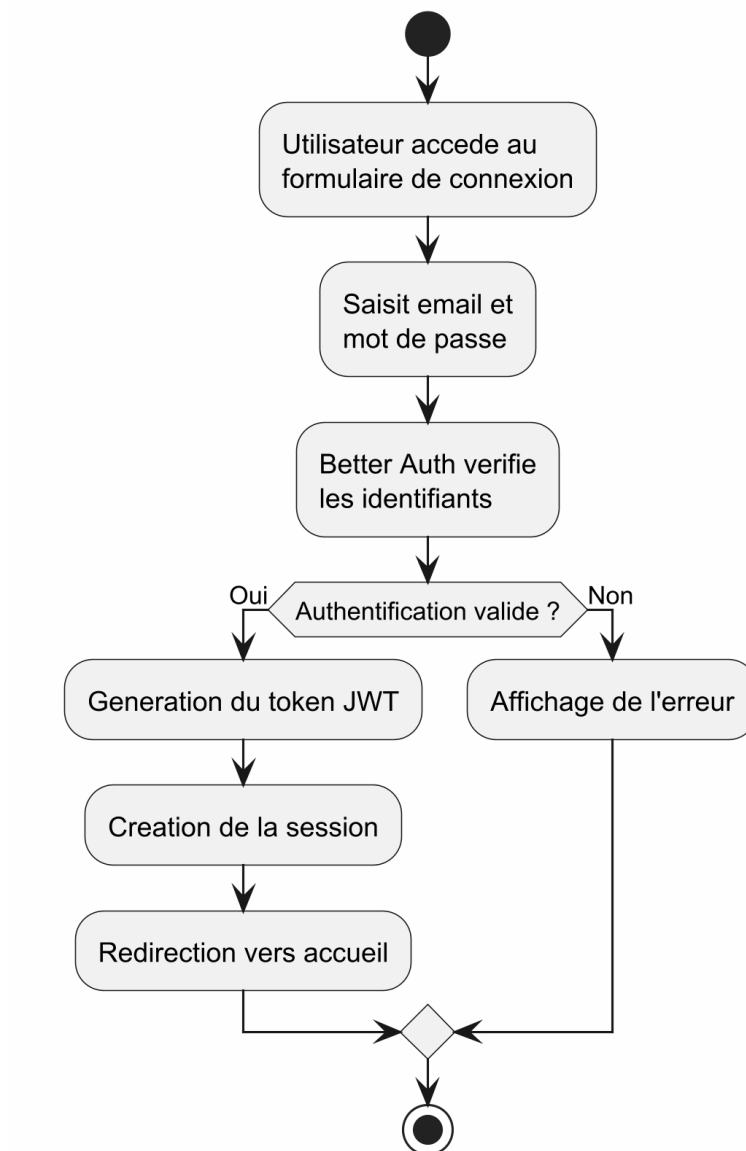
Diagramme d'Activite --- Flux d'Authentification

FIGURE 8 – Diagramme d'Activité — Flux d'Authentification

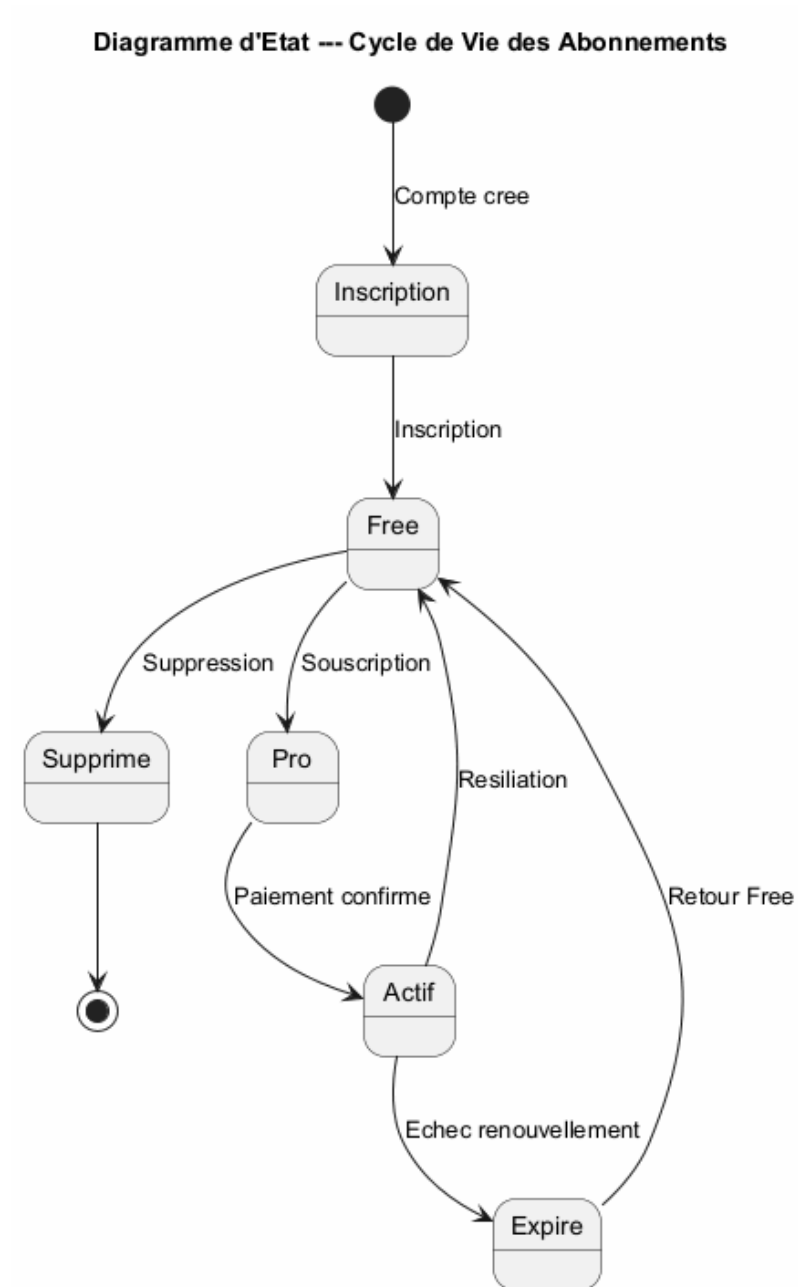


FIGURE 9 – Diagramme d'État — Cycle de Vie des Abonnements

rectory. Le framework Scrum a structuré le développement en sprints itératifs avec des livrables fonctionnels testables. La méthode Merise a fourni le MCD et le MLD qui décrivent la structure des données indépendamment de toute technologie. Les diagrammes UML complémentaires (cas d'utilisation, séquence, activité, état) documentent les aspects dynamiques du système. Cette combinaison de méthodes assure une documentation complète et cohérente du projet.

Introduction. Les normes sont des apports documentés contenant des spécifications techniques destinées à être utilisées systématiquement comme règles ou lignes directrices. Ce chapitre présente les normes internationales (ISO 9000) et professionnelles (IEEE) applicables au Génie Logiciel, ainsi que leur application concrète au projet SaaS Directory. La normalisation garantit la qualité, la traçabilité et la répétabilité des processus de développement.

5.1 La Norme ISO et la Certification ISO 9000

L'International Organization for Standardization (ISO), créée en 1947, est une fédération mondiale d'organismes nationaux de normalisation. La norme ISO 9000 traduit la démarche qualité au niveau de l'entreprise. Elle repose sur huit principes de management de la qualité : orientation client, leadership, implication du personnel, approche processus, management systémique, amélioration continue, approche factuelle, et relations mutuellement bénéfiques avec les fournisseurs.

Ces principes, bien que formulés pour des organisations industrielles et commerciales, trouvent une application directe dans le développement logiciel académique. L'**orientation client** se traduit par le respect des exigences fonctionnelles définies en début de projet (catalogue, authentification, messagerie). L'**approche processus** est matérialisée par le cycle de vie SDLC adopté (analyse, conception, développement, tests, déploiement, maintenance). L'**amélioration continue** est illustrée par les itérations successives du schéma de base de données (4 migrations Drizzle de 0000 à 0003) et les corrections apportées suite aux validations croisées au sein de l'équipe.

Dans le contexte de SaaS Directory, la gestion documentaire est assurée par la documentation du projet (README à la racine du repository, conventions de développement, schémas d'architecture dans le rapport), et la traçabilité est garantie par l'historique Git et les migrations Drizzle numérotées. Chaque migration constitue un artefact traçable qui peut être appliquée ou revertée, assurant la cohérence du schéma entre les environnements de développement. L'approche factuelle est démontrée par l'utilisation systématique de données mesurées (benchmark Redis, nombre de lignes de code, couverture de tests) pour orienter les décisions techniques.

5.2 Les Normes IEEE

L’Institute of Electrical and Electronics Engineers (IEEE) a défini un ensemble de normes s’intéressant aux processus, produits et techniques de base du génie logiciel. Ces normes constituent des références incontournables pour la profession.

Fixe la signification des termes et choisit les processus prioritaires du cycle de vie du logiciel. Peut être considérée comme un dictionnaire et une liste de vérification pour la gestion des projets.

Recommandée pour les spécifications des exigences du logiciel. Pourrait être intégrée avec les approches de validation par prototype.

Pour appliquer la validation et la vérification. Norme facilement adaptable à tous les problèmes.

Ces normes ont guidé la rédaction des spécifications de SaaS Directory. La norme 12207.0 a servi de référence pour la structuration du cycle de vie (SDLC). La norme 830 a inspiré la rédaction des user stories et du cahier des charges fonctionnel. La norme 1028 a guidé la stratégie de tests (validation des fonctionnalités, vérification de la conformité aux besoins).

5.3 Normalisation Appliquée au Projet

5.3.1 Normes de Codage

Le projet respecte des conventions strictes : PascalCase pour les composants React, camelCase pour les fonctions, et kebab-case pour les routes API. Les imports sont organisés par groupe : React/Next.js, bibliothèques tierces, composants internes, utilitaires. Ces conventions garantissent la lisibilité du code et facilitent les revues de code.

5.3.2 Architecture Normalisée

L’architecture suit le pattern MVC adapté à Next.js 16.2.4. Le **Modèle** est constitué par le schéma Drizzle ORM. La **Vue** est implémentée par les composants React. Le **Contrôleur** est assuré par les actions serveur du framework. Cette séparation respecte le principe de responsabilité unique et facilite les tests unitaires.

5.3.3 Gestion Documentaire et Traçabilité

La gestion documentaire est assurée par plusieurs artefacts : documentation de présentation et installation, conventions de développement, gestion des dépendances avec versions exactes (package.json), et messages de commit suivant une convention descriptive. La traçabilité est garantie par le versioning Git (branche principale, branches de fonctionnalités, validation croisée du code au sein de l'équipe) et les migrations Drizzle versionnées.

Conclusion du chapitre. Ce chapitre a présenté les normes ISO et IEEE applicables au Génie Logiciel, ainsi que leur application concrète au projet SaaS Directory. La normalisation des conventions de codage, de l'architecture MVC, et de la gestion documentaire assure la qualité, la lisibilité et la maintenabilité du codebase sur le long terme.

Introduction. Les tests logiciels constituent une phase critique du cycle de vie du logiciel, visant à détecter les erreurs et à garantir que le système répond aux exigences spécifiées. Ce chapitre présente les types de tests appliqués au projet SaaS Directory, la stratégie de test adoptée (pyramide des tests), les résultats du benchmark de performance, et les outils utilisés.

6.1 Types de Tests et Stratégie

La stratégie de tests de SaaS Directory repose sur une pyramide de tests adaptée au contexte du projet. Contrairement à la pyramide classique à trois niveaux, le projet ne compte pas de tests d'intégration distincts ; les tests sont organisés en deux niveaux principaux.

6.1.1 Tests Unitaires

Les tests unitaires sont implémentés avec Vitest 4.1.5. Dix fichiers de test couvrent l'authentification, la souscription, le schéma de base de données, les actions serveur, et les composants UI. Ces tests vérifient le comportement de chaque unité de code (fonction, composant) de manière isolée. L'utilisation de React Testing Library permet de tester les composants React de manière orientée utilisateur, en simulant les interactions (clics, saisies) et en vérifiant le rendu final.

Les tests unitaires couvrent les cas nominaux (parcours normal) et les cas limites (valeurs nulles, chaînes vides, dépassements de limites). La couverture cible est de 70% minimum, avec un focus sur les fonctions critiques : authentification, marketplace, messagerie, et abonnements.

6.1.2 Tests End-to-End

Les tests end-to-end sont réalisés avec Playwright 1.59.1. La configuration définit un projet Chromium avec tracing activé. Un fichier de test end-to-end valide le parcours utilisateur critique sur la page d'accueil. Ces tests simulent un utilisateur réel naviguant dans l'application, vérifiant que l'ensemble des composants fonctionne correctement en interaction.

Le parcours testé comprend : chargement de la page d'accueil, affichage du catalogue, navigation vers la page de connexion, et vérification des éléments UI essentiels. Les tests E2E complètent les tests unitaires en détectant les problèmes d'intégration entre modules.

6.1.3 Pyramide des Tests

La figure 10 illustre la stratégie de tests de SaaS Directory.

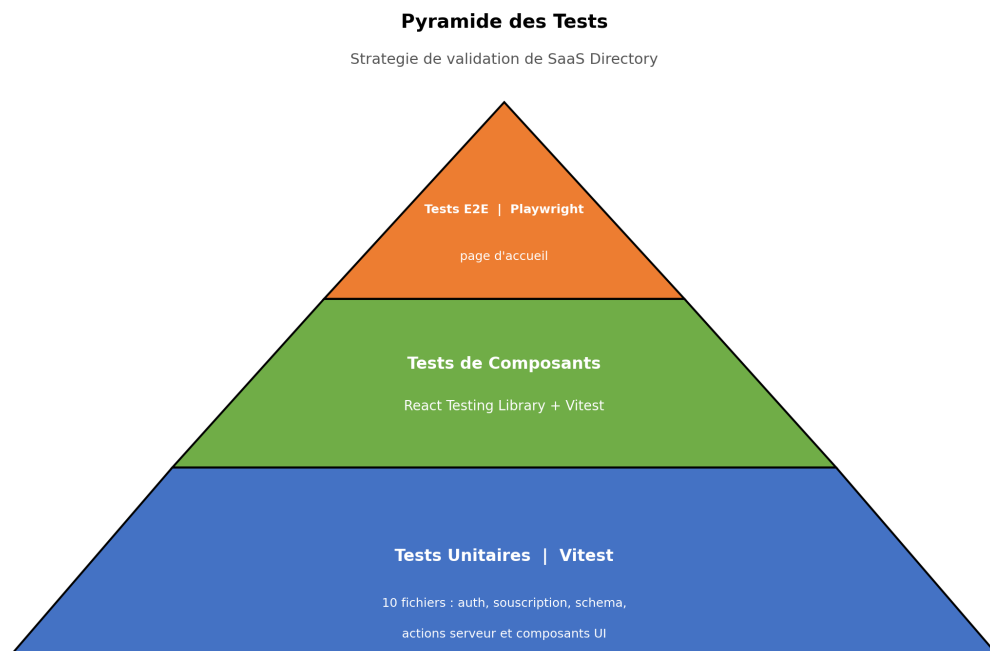


FIGURE 10 – Pyramide des Tests — Stratégie de validation de SaaS Directory

La base de la pyramide (large) représente les 10 fichiers de tests unitaires couvrant l'authentification, la souscription, le schéma de données, les actions serveur et les composants UI via Vitest. La couche intermédiaire regroupe les tests de composants React avec React Testing Library et Vitest. Le sommet (étroit) représente le fichier de test end-to-end Playwright ciblant la page d'accueil. Cette structure reflète la réalité du projet : une couverture unitaire dense au niveau des fonctions et des composants, et une couverture E2E ciblée sur les parcours critiques.

6.1.4 Tests de Performance

La performance de SaaS Directory a été évaluée par benchmark comparatif mesurant l'impact du cache Redis sur les opérations de lecture fréquentes. Le test a été réalisé le 10 mai 2026 via le script de benchmark qui compare les temps de réponse avec et sans cache distribué.

Architecture du cache. Le système utilise le pattern *Cache-Aside* (Lazy Loading) avec un client Redis encapsulé dans le module de cache. Chaque opération dispose d'un TTL configuré : 5 minutes pour la liste des produits, 30 minutes pour les catégories, 10 minutes pour le détail d'un produit, et 3 minutes pour la liste des publications. L'invalidation est gérée en écriture (Write-Through) : toute mutation supprime les clés affectées du cache.

Résultats mesurés. Le tableau 1 présente les temps de réponse en millisecondes pour trois opérations critiques.

TABLE 1 – Résultats du benchmark Cache Redis — SaaS Directory (10 mai 2026)

Opération	Sans Cache (ms)	Cache Warm (ms)	Gain
Liste des produits	9.17	0.05	99.4%
Catégories	1.69	0.07	95.6%
Liste des publications	1.88	0.06	97.0%

La figure 11 illustre la comparaison visuelle des temps de réponse. Le cache warm réduit les temps de réponse de plus de 95%.

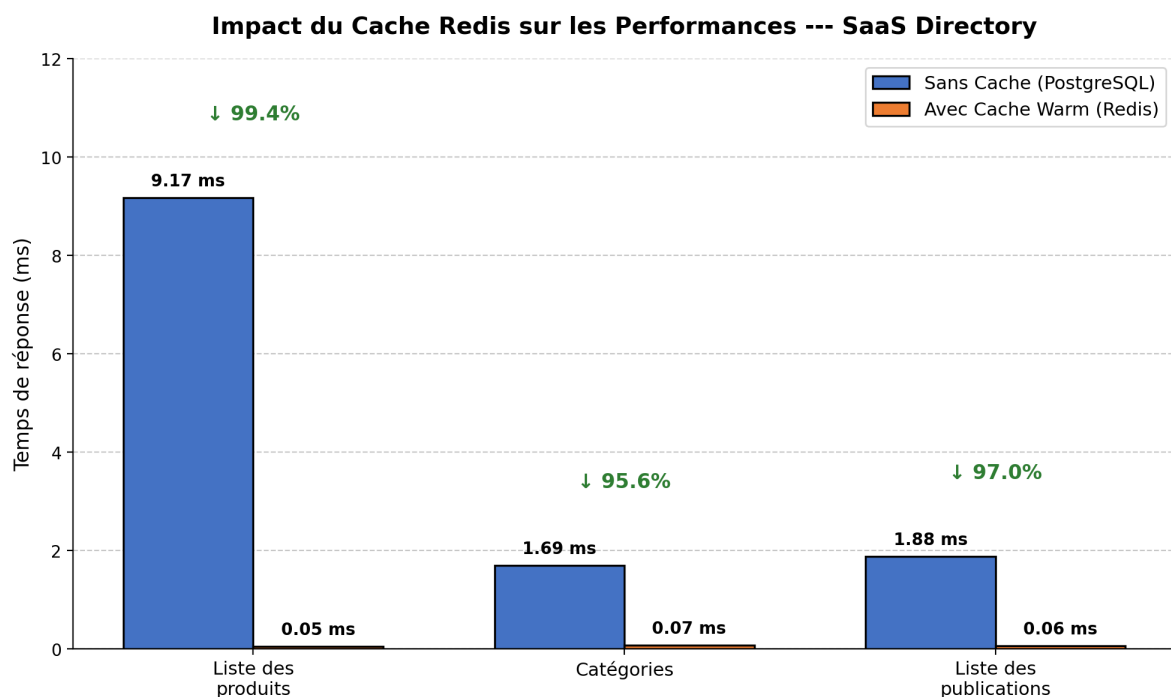


FIGURE 11 – Impact du Cache Redis sur les Performances — Comparaison avant/après

6.2 Outils et Environnement de Test

L'environnement de test utilise une base PostgreSQL dédiée sur le port 5433, séparée de l'environnement de développement principal. La séparation des environnements (développement, test, production) est assurée par des variables d'environnement dédiées, définies dans un fichier d'environnement non versionné. **Vitest** fournit un runner de tests rapide avec hot reload, ce qui permet aux développeurs de voir instantanément l'impact de leurs modifications sur les résultats des tests. Cette boucle de feedback rapide est essentielle dans un contexte académique où le temps de développement est limité.

Playwright offre une exécution de navigateurs réels (Chromium, Firefox, WebKit) pour les tests end-to-end, ce qui garantit que les interactions utilisateur fonctionnent correctement dans des conditions proches de la réalité. La configuration de Playwright spécifie une résolution d'écran de 1280x720 pour les tests desktop et 390x844 pour les tests mobile, couvrant ainsi les deux principaux modes d'accès.

Le choix de Vitest s'est imposé pour sa compatibilité native avec TypeScript et son intégration transparente avec Next.js 16.2.4, éliminant le besoin de configuration supplémentaire requis par Jest. Bien qu'il n'ait pas été mis en place de serveur d'intégration continue (CI) externe faute d'infrastructure dédiée, la pratique de validation manuelle des tests avant chaque session de travail collaboratif a maintenu une qualité satisfaisante.

6.3 Couverture et Résultats

La couverture de tests cible les fonctions critiques : authentification (création de compte, connexion), gestion des rôles, marketplace (création de produit, vote, commentaire), messagerie (envoi, réception), et abonnements (souscription, changement de plan). Les seuils de couverture sont configurés avec un minimum de 70%.

Conclusion du chapitre. Ce chapitre a présenté la stratégie de tests de SaaS Directory, organisée autour d'une pyramide à deux niveaux (unitaires + E2E). Les tests unitaires avec Vitest assurent la qualité du code à granularité fine, tandis que les tests end-to-end avec Playwright valident les parcours utilisateur critiques. Le benchmark de performance démontre l'impact significatif du cache Redis, avec des gains de plus de 95% sur les lectures fréquentes.

Introduction. La gestion des projets logiciels vise à structurer méthodiquement une réalité à venir, en appliquant un objectif et des actions à entreprendre avec des ressources données. Ce chapitre présente l'organisation du projet (WBS, PBS, OBS), la planification temporelle (diagramme de GANTT sur 60 jours), et l'estimation des coûts par la méthode COCOMO.

7.1 Organisation du Projet (WBS, PBS, OBS)

Trois outils structurants ont guidé l'organisation du projet, bien que leur formalisation ait été légère en raison du contexte académique et de la taille réduite de l'équipe. Ces outils, hérités des pratiques industrielles de gestion de projet, ont été adaptés pour rester opérationnels sans créer de surcharge bureaucratique inutile dans un groupe de trois personnes.

Le **Work Breakdown Structure (WBS)** décompose le projet en tâches hiérarchiques : analyse des besoins, conception (architecture, MCD/MLD, choix des technologies), développement des fondations (Next.js, Drizzle, authentification Better Auth), développement des fonctionnalités core (marketplace, messagerie, abonnements), tests (unitaires, E2E, performance), préparation au déploiement (Docker Compose pour environnement local), et documentation (rapport académique). La phase de conception n'a pas inclus de maquettes formelles (wireframes) en raison du temps limité ; les décisions d'interface ont été prises directement lors du développement des composants React.

Le **Product Breakdown Structure (PBS)** identifie les livrables tangibles du projet : authentification fonctionnelle avec gestion des rôles (user, admin), profil utilisateur opérationnel avec avatar et préférences, marketplace avec votes et commentaires, système de messagerie avec notifications en temps réel, abonnements Free/Pro avec limitation de fonctionnalités, documentation technique dans le rapport, et rapport académique complet avec 9 chapitres. Chaque élément du PBS correspond à une fonctionnalité vérifiable, ce qui facilite le suivi de l'avancement.

Le **Organizational Breakdown Structure (OBS)** répartit les responsabilités entre les trois développeurs du projet. Chaque membre a contribué au frontend (composants React/shadcn/ui), au backend (Server Actions, Drizzle ORM), et à la base de données (schéma, migrations) selon ses compétences et les besoins du moment. Cette flexibilité organisationnelle, bien que moins formelle qu'un OBS industriel, a permis d'optimiser l'utilisation des compétences disponibles sans rigidifier les rôles.

7.2 Planification (GANTT)

La figure 12 présente une planification hypothétique à visée pédagogique sur une échelle de 60 jours, illustrant le découpage type d'un projet web de cette envergure. Les 7 phases couvrent l'ensemble du cycle de vie SDLC.

Phase 1 : Analyse des besoins (jours 1–8). Spécifications fonctionnelles, user stories, identification des acteurs et des contraintes. Production du Product Backlog initial.

Phase 2 : Conception (jours 9–18). Architecture technique, modélisation Merise (MCD et MLD), choix de la stack (Next.js, Drizzle, PostgreSQL, Redis), définition des composants UI réutilisables.

Phase 3 : Développement — Fondations (jours 19–28). Initialisation du projet, configuration

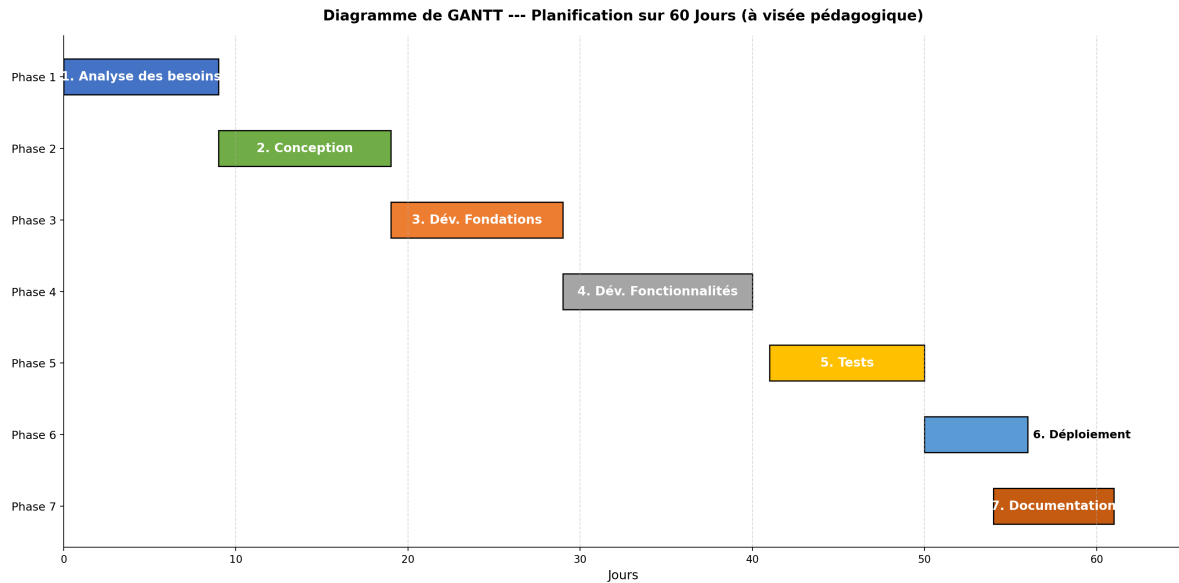


FIGURE 12 – Diagramme de GANTT — Planification sur 60 Jours (à visée pédagogique)

ORM, authentification Better Auth, base de données avec migrations.

Phase 4 : Développement — Fonctionnalités Core (jours 29–40). Marketplace (catalogue, votes, commentaires), messagerie en temps réel, système d’abonnements Free/Pro.

Phase 5 : Tests (jours 41–48). Tests unitaires Vitest, tests end-to-end Playwright, benchmark performance Redis, correction des anomalies.

Phase 6 : Préparation au Déploiement (jours 49–54). Configuration Docker Compose pour environnement local, vérification des services (Next.js, PostgreSQL, Redis), documentation des procédures de mise en production future.

Phase 7 : Documentation et Livrables (jours 55–60). Rédaction du rapport académique, documentation technique, manuel utilisateur, préparation de la soutenance.

7.3 Estimation des Coûts par COCOMO

L’estimation des coûts d’un projet logiciel est un exercice délicat. Pour contrer les biais classiques (optimisme, négligence technique), nous avons appliqué la méthode COCOMO (CONstructive COSt MOdel) de Barry Boehm.

7.3.1 Taille du Projet

Le codebase de SaaS Directory compte environ 11 000 lignes de code réparties sur 108 fichiers TypeScript/TSX, soit approximativement 11 KLOC à des fins d’estimation pédagogique. Répartition principale : application (2 100 LOC), composants (5 000 LOC), utilitaires serveur et bibliothèques (2 800 LOC), tests unitaires et d’intégration (1 000 LOC), tests end-to-end (100 LOC).

7.3.2 Application du COCOMO de Base

Pour un projet web moderne (Next.js 16.2.4, React 19.2.4, TypeScript 5), nous avons retenu le mode semi-détaché. Les coefficients sont $a = 3,0$ et $b = 1,12$.

TABLE 2 – Estimation COCOMO de Base — Mode Semi-Détaché (14 KLOC)

Paramètre	Formule / Valeur	Résultat
Taille du projet	Environ 11 000 LOC $\approx$ 11 KLOC	11 KLOC
Mode de développement	Semi-Détaché	$a = 3,0$; $b = 1,12$
Effort estimé	$E = a \times (\text{KLOC})^b$	$E \approx 58$ Personnes-Mois
Durée estimée	$D = 2,5 \times E^{0,35}$	$D \approx 9,5$ mois
Taille d'équipe théorique	$N = E/D$	$N \approx 6$ personnes
Taille d'équipe réelle	3 développeurs sur 3 semaines	2,3 Personnes-Mois

Ces chiffres constituent une estimation théorique à visée pédagogique. En réalité, le projet a été réalisé par trois développeurs sur environ 2,3 Personnes-Mois (17 jours de développement), illustrant l'écart fréquent entre les modèles prédictifs et la réalité des projets académiques intensifs.

Conclusion du chapitre. Ce chapitre a présenté la gestion de projet de SaaS Directory. Les structures WBS, PBS et OBS ont organisé le travail en tâches, livrables et responsabilités. Le diagramme de GANTT sur 60 jours illustre une planification type pour un projet de cette envergure. L'estimation COCOMO démontre la faisabilité théorique tout en soulignant l'écart avec la réalité académique.

Introduction. L'analyse des risques est une composante essentielle de la conduite de projet. Elle vise à identifier, évaluer et traiter les événements incertains qui pourraient affecter négativement les objectifs. Ce chapitre présente l'identification des risques du projet SaaS Directory, les stratégies de mitigation mises en oeuvre, et le plan de continuité.

8.1 Identification des Risques

Trois catégories principales de risques ont été identifiées.

8.1.1 Risques Techniques

Le risque technique majeur réside dans la complexité de Next.js 16.2.4 avec le App Router. Ce framework introduit des breaking changes significatifs par rapport aux versions précédentes. La do-

cumentation officielle a été consultée régulièrement pour anticiper ces changements. Un risque secondaire concerne la configuration des relations et indexes Drizzle ORM 0.45.2 avec PostgreSQL, qui nécessite une expertise SQL approfondie pour garantir les performances. La complexité des requêtes avec jointures multiples (launch + user + category + upvote) représente un risque de performance si les indexes ne sont pas correctement définis.

8.1.2 Risques de Sécurité

Les risques de sécurité concernent la protection des données utilisateurs et la prévention des attaques. L'injection SQL est mitigée par l'utilisation de Drizzle ORM avec des requêtes paramétrées. Les risques liés à l'authentification incluent la fuite de tokens JWT et la compromission des sessions. Better Auth 1.6.9 gère nativement plusieurs de ces menaces (expiration des tokens, rotation, rate limiting). Un risque supplémentaire concerne la gestion des mots de passe : bien que Better Auth hache les mots de passe avec bcrypt, une mauvaise configuration des variables d'environnement pourrait exposer des secrets.

8.1.3 Risques de Performance

Le marketplace doit supporter un grand nombre de produits sans dégradation du temps de réponse, ce qui nécessite l'optimisation des indexes PostgreSQL et la pagination côté serveur. Sans cache, les requêtes de liste (produits, catégories, publications) deviennent de plus en plus lentes à mesure que le volume de données croît. Le cache Redis est utilisé pour réduire la charge sur la base de données pour les données fréquemment accédées.

8.2 Analyse et Stratégies de Mitigation

8.2.1 Mitigation des Risques Techniques

La mitigation des risques techniques repose sur trois piliers complémentaires. Le premier pilier est la **veille technologique** : la documentation officielle de Next.js et de Better Auth a été consultée régulièrement pour anticiper les changements de comportement entre versions. Cette veille a été particulièrement utile lors de l'adoption du App Router de Next.js 16.2.4, qui introduit des patterns de programmation différents du pages router. Le deuxième pilier est l'**automatisation des tests** : les tests end-to-end Playwright 1.59.1 détectent les régressions fonctionnelles (authentification, marketplace, messagerie) à chaque modification du code, garantissant que les fonctionnalités existantes ne sont pas compromises par les nouveaux développements. Le troisième pilier est la **modularité de l'architecture** : le remplacement d'une technologie spécifique (par exemple le passage d'une autre bibliothèque d'authentification à Better Auth) est facilité par l'encapsulation des dépendances dans des modules dédiés (auth.ts, db.ts, cache.ts). Les migrations Drizzle versionnées (0000 à 0003) offrent en outre un mécanisme de retour en arrière en cas de problème de schéma.

8.2.2 Mitigation des Risques de Sécurité

La mitigation des risques de sécurité est assurée par plusieurs mécanismes en profondeur. **Better Auth 1.6.9** gère nativement la sécurité des sessions (tokens JWT avec expiration, rotation automatique, rate limiting sur les endpoints d'authentification), ce qui réduit la surface d'attaque sans nécessiter de développement spécifique. Les **Server Actions** valident systématiquement les entrées utilisateur via Zod 4.4.3 avant toute interaction avec la base de données, ce qui prévient l'injection SQL même si Drizzle ORM utilise déjà des requêtes paramétrées. Les **variables d'environnement** sensibles (clés API, secrets d'authentification, URL de base de données) sont stockées dans des fichiers `.env` explicitement exclus du versioning Git (fichier `.gitignore`), garantissant que ces secrets ne sont jamais exposés dans le repository public. Un risque résiduel demeure : la compromission d'une variable d'environnement par un membre de l'équipe lors d'un transfert de fichier. Ce risque a été mitigué par la sensibilisation aux bonnes pratiques de sécurité.

8.2.3 Mitigation des Risques de Performance

La mitigation des risques de performance s'appuie sur trois techniques. Premièrement, les **indexes PostgreSQL** définis dans le schéma Drizzle (indexes sur `makerId`, `slug`, `isForSale`, `isFeatured` pour la table `Launch`) accélèrent les requêtes de filtrage et de recherche les plus fréquentes. Deuxièmement, le **cache Redis** stocke les données fréquemment accédées (sessions utilisateur, profils, listes de catégories) en mémoire, évitant des lectures répétées sur le disque de la base de données. Le benchmark de performance a mesuré des gains de plus de 95% sur les lectures fréquentes grâce à ce cache. Troisièmement, la **pagination côté serveur** pour le marketplace et la messagerie limite le nombre de lignes transférées à chaque requête, garantissant des temps de réponse constants indépendamment du volume total de données.

8.3 Plan de Continuité

En cas d'incident majeur, plusieurs mesures sont prévues : restauration depuis les backups PostgreSQL, reconstruction du schéma via les migrations Drizzle (0000 à 0003), retour à une version stable via Git (branche principale et tags), et documentation technique facilitant l'onboarding d'un nouveau développeur. La modularité du code permet de remplacer un composant défaillant sans impacter l'ensemble du système.

Conclusion du chapitre. Ce chapitre a identifié et analysé les risques techniques, sécuritaires et de performance du projet SaaS Directory. Les stratégies de mitigation (veille technologique, tests automatisés, cache, validation des entrées) et le plan de continuité (backups, migrations, versioning) garantissent la résilience du système face aux incidents.

Introduction. La conduite du changement est la discipline qui vise à accompagner les transformations au sein d'un projet. Ce chapitre présente les différents types de changements survenus pendant le développement de SaaS Directory, la gestion des versions via Git et les migrations Drizzle, et les mécanismes d'adaptation et de communication mis en place.

9.1 Nature des Changements

Plusieurs types de changements ont affecté le projet.

Les **changements technologiques** incluent l'ajout de Framer Motion pour les animations, l'intégration de Redis pour le cache côté serveur, et l'évolution du schéma Drizzle avec les migrations successives. L'ajout de Redis est particulièrement significatif : il est survenu en milieu de projet après l'identification de problèmes de performance lors des tests, illustrant l'adaptation agile du projet.

Les **changements fonctionnels** concernent l'ajout de fonctionnalités non prévues initialement : système de notifications, édition et suppression des commentaires, pagination infinie pour la messagerie, reset de mot de passe. Ces évolutions ont été intégrées dans les sprints successifs sans perturbation du calendrier global.

Les **changements organisationnels** touchent la répartition des tâches entre les trois développeurs. Au fur et à mesure de l'avancement, les responsabilités se sont adaptées en fonction des compétences émergentes et des blocages rencontrés. Par exemple, le développeur ayant le plus d'expérience avec Drizzle ORM a naturellement pris en charge les migrations complexes, tandis que celui le plus à l'aise avec React a dirigé la refonte des composants UI. Cette flexibilité organisationnelle, bien que non formelle comme dans un cadre Scrum industriel, a permis d'optimiser l'utilisation des compétences disponibles sans rigidifier les rôles.

9.2 Gestion des Versions

Le contrôle de versions est assuré par Git avec une stratégie de branches simple adaptée à un contexte académique. La branche principale constitue la version stable du projet. Les branches de fonctionnalités isolent le développement de chaque incrément lorsque plusieurs développeurs travaillent simultanément sur des parties différentes du code. La qualité avant fusion est assurée par la validation croisée du code au sein de l'équipe, bien qu'il n'ait pas été mis en place de processus formel de pull requests sur une plateforme d'hébergement externe.

Les migrations Drizzle constituent un mécanisme de gestion du changement au niveau de la persistance. Chaque migration est un fichier SQL versionné (de 0000 à 0003) qui peut être appliquée et revertée. La figure 13 présente la chronologie de l'évolution du schéma.

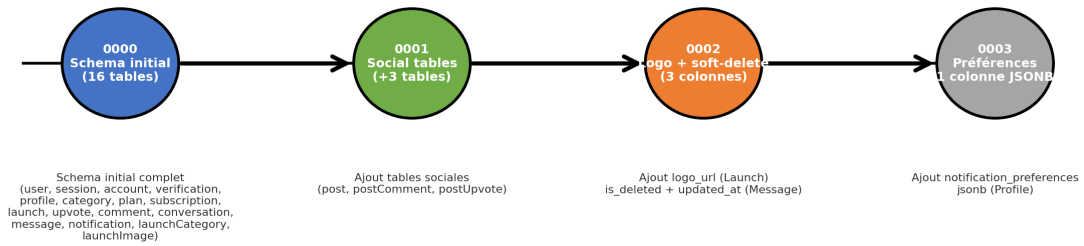


FIGURE 13 – Chronologie de l'Évolution du Schéma — Migrations Drizzle

La migration 0000 a créé le schéma initial avec 16 tables. La migration 0001 a ajouté les tables sociales (post, postComment, postUpvote). La migration 0002 a ajouté le champ logoUrl dans la table Launch et les champs isDeleted et updatedAt dans la table Message. La migration 0003 a ajouté les préférences de notification (JSONB) dans la table Profile.

9.3 Adaptation et Communication

L'architecture modulaire du projet a facilité l'adaptation aux changements. L'ajout du cache Redis n'a nécessité que la création du module cache.ts et la modification des actions serveur concernées, sans impact sur les composants UI. L'ajout des préférences de notification (migration 0003) n'a concerné que la table Profile et l'interface associée.

La communication autour des changements repose sur plusieurs canaux adaptés au contexte académique. La documentation technique est maintenue à jour dans le fichier README du projet et dans le rapport académique. Les messages de commit suivent une convention descriptive indiquant le type de modification (feat, fix, docs, refactor) et la nature du changement. Les discussions techniques entre développeurs, bien que non formalisées dans des revues de code structurées sur une plateforme externe, ont permis de valider les décisions d'architecture et de partager les connaissances sur les technologies utilisées. Des points de synchronisation informels, lors des sessions de travail collectif, permettaient d'aligner l'équipe sur les changements en cours et les obstacles rencontrés.

Conclusion du chapitre. Ce chapitre a présenté la conduite du changement du projet SaaS Directory. Les changements technologiques (Redis), fonctionnels (notifications, pagination) et organisationnels ont été gérés via le versioning Git, les migrations Drizzle, et la communication régulière de l'équipe. L'architecture modulaire et l'approche agile ont permis d'intégrer ces changements sans dégradation de la qualité ou des délais.

Ce rapport a présenté l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de SaaS Directory. Du point de vue du cycle de vie, le modèle SDLC adopté a permis de structurer le développement en phases claires : analyse des besoins, conception, développement, tests, déploiement et maintenance.

La démarche agile, matérialisée par une adaptation académique du framework Scrum, a assuré une progression itérative avec des livrables fonctionnels à chaque sprint. Le schéma Drizzle ORM de 20 tables et les composants React modulaires garantissent la maintenabilité et l'évolutivité du système. Les diagrammes Merise (MCD et MLD) fournissent une documentation structurée de la base de données, tandis que les diagrammes UML (cas d'utilisation, séquence, activité, état, déploiement, architecture en couches) illustrent les aspects dynamiques et statiques du système.

La gestion de projet, illustrée par le WBS, le diagramme de GANTT sur 60 jours, et l'estimation COCOMO, démontre la faisabilité d'une planification rigoureuse. L'analyse des risques a permis d'anticiper et de mitiger les principales menaces techniques et sécuritaires.

L'utilisation d'une stack technologique moderne et cohérente — Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, Better Auth 1.6.9, PostgreSQL, et Redis — a grandement facilité le développement et garanti la qualité du livrable final.

Les perspectives d'évolution de SaaS Directory sont nombreuses. L'intégration complète de Stripe pour le paiement (le schéma contient déjà les champs client et abonnement Stripe préparés pour cette évolution), l'ajout d'un système de recherche avancée avec indexation full-text PostgreSQL, le déploiement sur un hébergeur cloud (Vercel, AWS, ou Azure), et la mise en place d'une infrastructure de conteneurisation avec Docker pour la production constituent les axes de développement futurs envisagés.

- Barry Boehm, *Software Engineering Economics*, Prentice-Hall, 1977.
- Grady Booch, *Object-Oriented Analysis and Design with Applications*, 2nd Edition, Benjamin/Cummings, 1994.
- ISO/IEC 12207 :2017, Systems and software engineering — Software life cycle processes.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 1028-2008, IEEE Standard for Software Reviews and Audits.
- IEEE Std 12207.0-1996, IEEE Standard for Information Technology — Software life cycle processes.
- Cours de Génie Logiciel et Conduite de Projet, Année Universitaire 2025-2026.
- Next.js Documentation, <https://nextjs.org/docs>, consulté en 2026.
- React Documentation, <https://react.dev>, consulté en 2026.
- Drizzle ORM Documentation, <https://orm.drizzle.team>, consulté en 2026.
- Better Auth Documentation, <https://www.better-auth.com>, consulté en 2026.
- Tailwind CSS Documentation, <https://tailwindcss.com>, consulté en 2026.
- PostgreSQL Documentation, <https://www.postgresql.org/docs>, consulté en 2026.
- Redis Documentation, <https://redis.io/documentation>, consulté en 2026.
- Vitest Documentation, <https://vitest.dev>, consulté en 2026.
- Playwright Documentation, <https://playwright.dev>, consulté en 2026.

Annexe A : Dependances Principales

TABLE 3 – Dependances principales du projet SaaS Directory

Dependance	Version	Description
next	16.2.4	Framework React avec App Router
react	19.2.4	Bibliotheque UI
react-dom	19.2.4	Rendu DOM React
drizzle-orm	0.45.2	ORM type-safe pour PostgreSQL
better-auth	1.6.9	Authentification OAuth + 2FA
pg	8.20.0	Driver PostgreSQL
@tanstack/react-query	5.100.9	Gestion de cache et requetes
zod	4.4.3	Validation de schemas
framer-motion	12.38.0	Animations React
ioredis	5.10.1	Client Redis
resend	6.12.2	Service d'envoi d'emails
sonner	2.0.7	Notifications toast UI
tailwindcss	^4	Framework CSS utilitaire
typescript	^5	Typage statique
vitest	4.1.5	Tests unitaires et d'integration
eslint	^9	Linting du code
eslint-config-next	16.2.4	Regles ESLint Next.js

Annexe B : Diagramme de Deploiement

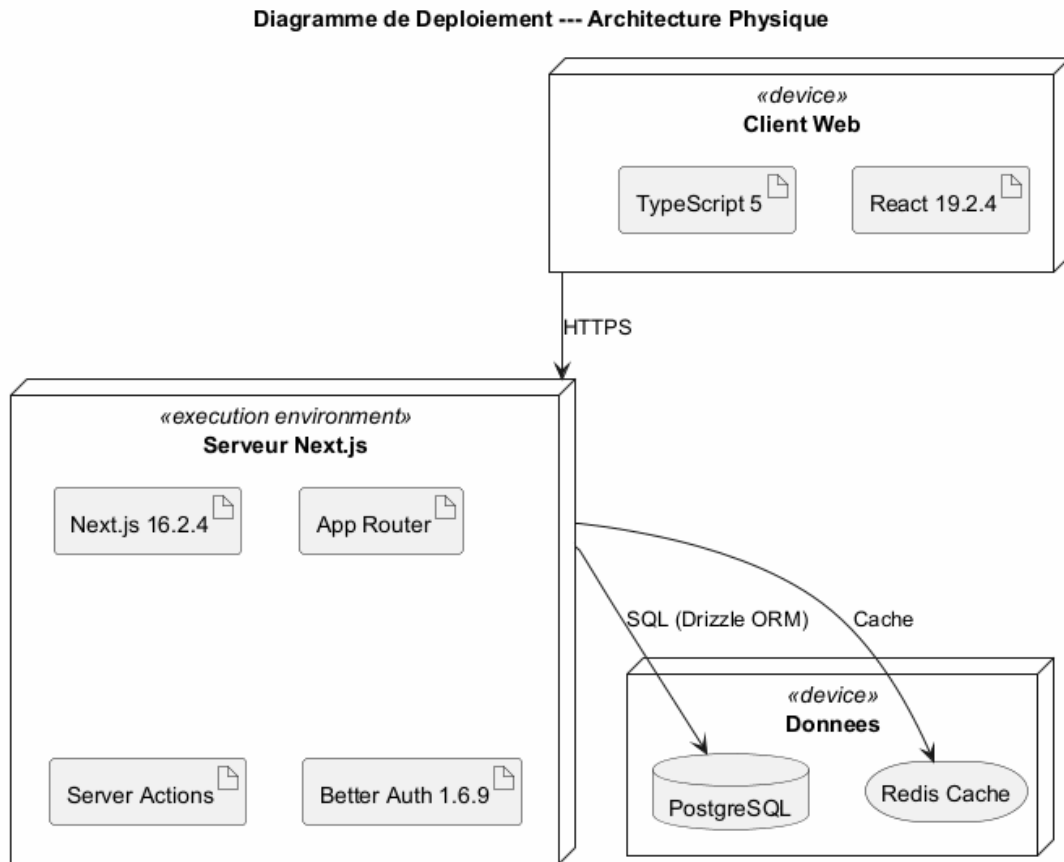


FIGURE 14 – Diagramme de Deploiement — Architecture Physique de SaaS Directory

La figure 14 décrit l'architecture physique du système. SaaS Directory repose sur une architecture distribuée sur trois environnements principaux : le client web (navigateur), la plateforme serveur (Next.js 16.2.4), et les services de données (PostgreSQL + Redis).

Le client web (React 19.2.4 / TypeScript 5) s'exécute dans le navigateur et communique avec les fonctions serverless via HTTPS. Les Server Actions Next.js 16.2.4 interagissent avec la base de données PostgreSQL via Drizzle ORM 0.45.2. Redis est utilisé comme cache côté serveur pour réduire la charge sur la base de données. L'authentification est assurée par Better Auth 1.6.9.